

PP WS20/21

Viral Simulator

Allgemeine Hinweise

- Alle Klassen finden Sie im Paket `de.uniwue.virusinf`.
- Das Paket befindet sich im Ordner `src`.
- Die Skelette der Klassen sind bereits gegeben, die dort eingefügten Exceptions sind durch den erwarteten Code zu ersetzen.
- Es ist gestattet, aber nicht nötig, neben den geforderten Klassen weitere Klassen oder zusätzliche Methoden zu implementieren.

Unit-Tests

- Es stehen lokale Unit-Tests zum Überprüfen Ihrer Implementierung zur Verfügung.
- Die Testklassen befinden sich im Ordner `test`.
- Jede Aufgabe in dieser Aufgabenstellung hat eine passende Testklasse (Name: `"Test<Klassenname>"`).
- Die Tests sind von Anfang an ausführbar.

Aufgaben

Das Bologna-Virus erobert die Welt. Zur Planung von Gegenmaßnahmen ist ein Wissen über die geographische Ausbreitung notwendig. Sie sind beauftragt eine Simulation zu implementieren, die diese vorhersagt.

Ihr Simulationmodell besteht dabei aus einer Menge von Städten, die (noch) gesunde und infizierte Bewohner besitzen. Zwischen Nachbarstädten existieren Verkehrsflüsse, über die Menschenmengen ausgetauscht werden. Die Simulation erfolgt in Schritten und berechnet in Abhängigkeit der Einwohner, des Verkehrs, sowie weiterer Modellparameter die Einwohnerzahlen für den nächsten Simulationsschritt.

Die Klausur ist in folgende Unteraufgaben gegliedert:

Nummer	Name	Punkte
1.	ParameterValidator	6
2.	SimulationParameters	12
3.	City	34
4.	InfectionBehaviorFactory	15
5.	Simulation	23
Summe		90

Das Erreichen von **60** Punkten ist hinreichend, aber nicht zwingend notwendig, um die Prüfung zu bestehen.

Wir wünschen viel Erfolg!

Die Klasse ParameterValidator

Im Verlauf der Aufgabe kommt es öfters vor, dass Integer darauf geprüft werden müssen, ob sie negativ oder nicht-positiv sind, z.B. um eine Anzahl an Menschen zu beschreiben, oder dass Floats darauf geprüft werden müssen, ob sie eine Zahl zwischen 0% und 100% speichern.

Die bereitgestellten Methoden sollen später folgendermaßen funktionieren:

```
public ExampleClass {
    private int positiveParam;

    public ExampleClass(int positiveParam) {
        this.positiveParam = ParameterValidator.throwIfNotPositive(positiveParam);
    }
}
```

Implementieren Sie die folgenden Methoden:

- `public static int throwIfNotPositive(int i, String msg)`
Falls `i` kleiner oder gleich 0 ist, soll eine `IllegalArgumentException` mit der Nachricht `msg` geworfen werden. Ansonsten soll `i` zurückgegeben werden.
- `public static int throwIfNegative(int i, String msg)`
Falls `i` kleiner als 0 ist, soll eine `IllegalArgumentException` mit der Nachricht `msg` geworfen werden. Ansonsten soll `i` zurückgegeben werden.
- `public static float throwIfNotPercentage(float f, String msg)`
Falls `f` kleiner als 0 oder größer als 1 ist, soll eine `IllegalArgumentException` mit der Nachricht `msg` geworfen werden. Ansonsten soll `f` zurückgegeben werden.

Die Klasse SimulationParameters

In dem zu implementierenden Simulationsmodell gibt es einige Variablen, die das Ausbreitungsverhalten des Virus beeinflussen können. Konkret gibt es den Parameter `infectionsPerIll`, der beschreibt, wieviele Neuinfektionen ein Infizierter in seiner Stadt in einem Simulationsschritt auslöst. Weiterhin beschreibt `percentageDie`, welcher Anteil der erkrankten Bevölkerung einer Stadt in einem Simulationsschritt dem Virus erliegt.

Objekte der Klasse `SimulationParameters` sollen dazu genutzt werden diese Parameter abzuspeichern.

Implementieren Sie dazu folgende Funktionalität:

- `public SimulationParameters(int infectionsPerIll, float percentageDie)`
Falls `infectionsPerIll` keine positive Zahl ist, soll eine `IllegalArgumentException` mit der Nachricht `"infectionPerIll has to be a positive Integer!"` geworfen werden.
Falls `percentageDie` keine gültige Prozentzahl ist, soll eine `IllegalArgumentException` mit der Nachricht `"percentageDie has to be between 0 and 1 (both inclusive)!"` geworfen werden.
- `public int getInfectionsPerIll()`
Gibt `infectionsPerIll` zurück.
- `public int getPercentageDie()`
Gibt `percentageDie` zurück.
- `public boolean equals(Object o)`
Überschreiben Sie diese Methode so, dass zwei `SimulationParameters`-Objekte gleich sind, wenn ihre `infectionsPerIll`- und `percentageDie`-Attribute gleich sind.

- `public int hashCode()`
Überschreiben Sie diese Methode so, dass sie den in der API festgelegten Vertrag zwischen `equals` und `hashCode` erfüllt.
- `public String toString()`
Überschreiben Sie diese Methode so, dass sie eine von Ihnen wählbare `String`-Darstellung des Objekts zurückgibt. Diese einzige Anforderung ist, dass die Rückgabe aus einem gültigen `String`-Objekt bestehen muss.

Die Klasse `City`

Eine `City` besitzt eine Anzahl an gesunden Bewohnern (`peopleHealthy`), eine Anzahl an erkrankten Bewohnern (`peopleIll`), sowie eine Menge an Nachbarstädten mit zugehörigem Verkehrsflussstärke (`trafficDensity`).

Diese Klasse ist auch für die Implementierung der Logik des Simulationsmodell zuständig, welche in der `update`-Methode zu finden ist.

Eine Besonderheit des Modells ist, dass jede Stadt ein eigenes `infectionBehavior` besitzt, das einmalig ausgelöst wird, wenn die Infizierten mehr als 10% der Stadtbevölkerung ausmachen.

Implementieren Sie folgenden Funktionalität:

- `public City(String name, int peopleHealthy, int peopleIll, Consumer<City> infectionBehavior)`
Falls `name` `null` ist, soll eine `NullPointerException` geworfen werden.
Falls `peopleHealthy` eine negative Zahl ist, soll eine `IllegalArgumentException` mit der Nachricht `"peopleHealthy cannot be negative!"` geworfen werden.
Falls `peopleIll` eine negative Zahl ist, soll eine `IllegalArgumentException` mit der Nachricht `"peopleIll cannot be negative!"` geworfen werden.
Falls `infectionBehavior` `null` ist, soll eine `NullPointerException` geworfen werden.
- `public City(String name, int people, Consumer<City> infectionBehavior)`
Dieser Konstruktor soll eine komplett gesunde Stadt mit `people` Einwohnern erstellen.
Es sollen die gleichen Exceptions wie beim andere Konstruktor geworfen werden.
- `public String getName()`
Gibt `name` zurück.
- `public int getPeopleHealthy()`
Gibt `peopleHealthy` zurück.
- `public int getPeopleIll()`
Gibt `peopleIll` zurück.
- `public int getPeopleAll()`
Gibt zurück, wieviele Bewohner sich insgesamt in der Stadt aufhalten.
- `public void setName(String name)`
Setzt `name` . Falls der neue Name `null` ist, soll eine `NullPointerException` geworfen werden.
- `public void setPeopleHealthy(int peopleHealthy)`
Setzt `peopleHealthy` . Falls `peopleHealthy` eine negative Zahl ist, soll eine `IllegalArgumentException` mit der Nachricht `"peopleHealthy cannot be negative!"` geworfen werden.
- `public void setPeopleIll(int peopleIll)`
Setzt `peopleIll` . Falls der neuwert eine negative Zahl ist, soll eine `IllegalArgumentException` mit der Nachricht `"peopleIll"` geworfen werden.

cannot be negative!" geworfen werden.

- `addNeighborCity(City newNeighbor, int trafficDensity)`

Fügt `newNeighbor` zu der Menge der Nachbarstädte hinzu und speichert ab, das dorthin ein Verkehrsfluss mit Stärke `trafficDensity` besteht.

Falls `newNeighbor` `null` ist, soll eine `NullPointerException` geworfen werden.

Falls `newNeighbor` die Stadt selbst ist, soll eine `IllegalArgumentException` mit der Nachricht "You cannot be your

own neighbor!" geworfen werden.

Falls `newNeighbor` bereits in der Menge der Nachbarstädte vorhanden ist, soll eine

`IllegalStateException` mit der Nachricht "The new neighbor already exists!" geworfen werden.

Falls `trafficDensity` keine positive Zahl ist, soll eine `IllegalArgumentException` mit der Nachricht "trafficDensity

has to be positive!" geworfen werden.

- `public Optional<Integer> getTrafficToNeighbor(City neighbor)`

Falls `neighbor` in der Menge der Nachbarstädte vorhanden ist, soll die `trafficDensity` zu diesem zurückgegeben werden. Sonst soll `Optional.empty` zurückgegeben werden.

- `public List<City> getNeighbors()`

Gibt die Menge der Nachbarstädte als Liste zurück.

- `public void update(int infectionsPerIll, float percentageDie)`

Implementieren Sie den folgenden Algorithmus.

Beachten Sie, dass Anzahlen immer als `int` zu speichern sind. Bei nicht-ganzzahligen Ergebnissen soll immer abgerundet werden.

Zuerst wird die Ausbreitung lokal in der Stadt berechnet:

- Berechnen Sie die Anzahl der Neuinfektionen als Produkt von `peopleIll` und `infectionsPerIll`.

Beachten Sie dabei, dass diese Anzahl höchstens so groß wie `peopleHealthy` sein kann.

- Verringern Sie `peopleIll`, sodass es nur noch $(1 - \text{percentageDie})$ viele Infizierte gibt.
- Addieren Sie die Zahl der Neuinfektionen zu `peopleIll`.
- Subtrahieren Sie die Zahl der Neuinfektionen von `peopleHealthy`.

Dann wird die Verbreitung durch Verkehr in andere Städte berechnet.

Folgende Schritte müssen für jede Nachbarstadt (genannt `n`) ausgeführt werden.

Die Nachbarn können in beliebiger Reihenfolge behandelt werden.

- Bestimmen Sie Anzahl der gesunden Reisenden. Im Regelfall beträgt diese `trafficDensity` von `n`, allerdings kann sie nicht größer sein als `peopleHealthy`.
- Bestimmen Sie Anzahl der kranken Reisenden. Im Regelfall beträgt diese `trafficDensity` von `n`, allerdings kann sie nicht größer sein als `peopleIll`.
- Subtrahieren Sie die Zahl der gesunden Reisenden von `peopleHealthy` der aktuellen Stadt.
- Addieren Sie die Zahl der gesunden Reisenden zu `peopleHealthy` von `n`.
- Subtrahieren Sie die Zahl der kranken Reisenden von `peopleIll` der aktuellen Stadt.
- Addieren Sie die Zahl der kranken Reisenden zu `peopleIll` von `n`.

Prüfen Sie am Ende, ob das `infectionBehavior` auszuführen ist (dies gehört nicht mehr zu "müssen für jede Nachbarstadt ausgeführt werden").

Stellen Sie dazu Folgendes sicher:

- Dass das `infectionBehavior` bisher noch nicht ausgeführt wurde.
- Dass es noch Bewohner in der Stadt gibt.
- Dass der Anteil der Kranken mehr als 10% der Gesamtbevölkerung ausmacht (Vorsicht, keine Integerdivision verwenden).

Sind die Anforderungen erfüllt, soll die `accept`-Methode des `infectionBehavior` aufgerufen werden.

- `public String toString()`

Überschreiben Sie diese Methode so, dass sie folgenden `String`-Darstellung zurückgibt:

Die erste Zeile soll lauten:

```
<name> (<peopleIll> people of <peopleHealthy + peopleIll> infected)
```

Nutzen Sie dann `System.lineSeparator()` für einen Zeilenumbruch. Falls die Stadt keine Nachbarn besitzt, soll die nächste (und letzte Zeile) der Rückgabe sein:

```
Neighbors: -
```

Gibt es Nachbarn, sollen diese in folgendem Format aufgelistet werden.

```
Neighbors:
  <neighbor1.name> (<trafficDensity_to_neighbor1>)
  <neighbor2.name> (<trafficDensity_to_neighbor2>)
  ...
  <neighborN.name> (<trafficDensity_to_neighborN>)
```

Die Nachbarstädte können in beliebiger Reihenfolge ausgegeben werden.

Benutzen Sie `System.lineSeparator()` für Zeilenumbrüche und `\t` für Einrückungen.

Am Ende des `String` soll sich kein Zeilenumbruch mehr befinden.

Die Klasse `InfectionBehaviorFactory`

Jede `City` besitzt ein `infectionBehavior`, das bei einem Infiziertenanteil von mehr als 10% ausgeführt wird. Die `InfectionBehaviorFactory` bietet einige statische Methoden, um häufige Verhaltensmuster leicht zu erzeugen.

Die zu implementierenden Methoden decken die folgenden drei typischen Szenarien ab:

1. Um die lokale Ausbreitung des Virus in der Stadt zu stoppen, wird die Bevölkerung präventiv durch das humanoide Equivalent von Alt+F4 auf einen Bruchteil reduziert.
2. Um die globale Ausbreitung des Virus zu verhindern, werden die Verkehrsflüsse zu Nachbarstädten unterbunden.
3. Um den Namen einer einst prachtvollen Stadt nicht in zukünftigen Geschichtsbüchern zu entwürdigen, benennt sich eine Stadt im Infektionsfall kurzerhand um.

Erzeugen Sie diese Verhaltensmuster in den folgenden Klassenmethoden:

- `public static Consumer<City> createBehaviorReducePopulation(float survivorPercentage)`
Falls `survivorPercentage` keine gültige Prozentzahl ist, soll eine `IllegalArgumentException` mit der Nachricht "survivorPercentage has to be between 0 and 1 (both inclusive)!" geworfen werden. Wird `0.5f` als Parameter übergeben, soll "Balanced, as all things should be!", gefolgt von einem Zeilenumbruch, auf `System.out` ausgegeben werden. Schließlich soll ein `Consumer` zurückgegeben werden, dessen `accept`-Methode die gesunde und kranke Bevölkerungszahl der übergebenen Stadt auf `survivorPercentage` der jeweiligen aktuellen Anzahl reduziert (nicht-ganzzahlige Ergebnisse sollen abgerundet werden).
- `public static Consumer<City> createBehaviorCutAllTraffic()`
Geben Sie einen `Consumer` zurück, dessen `accept`-Methode alle Städte aus der Menge der Nachbarstädte entfernt.
- `public static Consumer<City> createBehaviorChangeName(String name)` Falls `name` `null` ist, werfen Sie eine `NullPointerException`. Falls `name` ein leerer `String` ist, werfen Sie eine `IllegalArgumentException` mit der Nachricht "name

cannot be
empty" .

Ansonsten geben Sie einen `Consumer` zurück, dessen `accept`-Methode den Namen der übergebenen Stadt zu `name` ändert.

Die Klasse `Simulation`

Instanzen dieser Klasse verwalten eine komplette Simulation.

Dazu benötigen sie eine Menge an möglicherweise untereinander verbundenen Städten und einen Satz `Simulationparameter`.

Implementieren Sie die folgende Funktionalität:

- `public Simulation(List<City> cities, SimulationParameters params)`
Falls `cities` `null` ist oder eines der Listenelemente `null` ist, soll eine `NullPointerException` geworfen werden.
Falls `cities` leer ist, soll eine `IllegalArgumentException` mit der Nachricht "No cities - no simulation!" geworfen werden. Falls `params` `null` ist, soll eine `NullPointerException` geworfen werden.
- `public List<City> citiesSortedAlphabetically()` Liefert eine Liste mit den in der Simulation befindlichen Städte, alphabetisch sortiert nach den Städtenamen.
- `public int getStepCounter()`
Gibt zurück, wieviele Simulationsschritte schon ausgeführt wurden. Für eine neu erstellte Simulation soll also 0 geliefert werden.
- `public void simulateStep()`
Ruft die `update`-Methode jeder Stadt in `cities` auf. Da unterschiedliche Reihenfolgen zu verschiedenen Ergebnissen führen, wird festgelegt, dass die Städte in alphabetischer Reihenfolge aktualisiert werden müssen. Verwenden Sie hierzu `infectionsPerIll` und `percentageDie` aus dem `SimulationParams`-Objekt `params`.
Zusätzlich soll der Schrittzähler inkrementiert werden.
- `public int totalPeopleHealthy()`
Gibt zurück, wieviele gesunden Menschen in allen Städten zusammen wohnen.
- `public int totalPeopleIll()`
Gibt zurück, wieviele kranke Menschen in allen Städten zusammen wohnen.
- `public int totalPeople()`
Gibt zurück, wieviele Menschen in allen Städten zusammen wohnen.
- `public String toString()`
Gibt eine `String`-Repräsentation eines `Simulation`-Objekts

```
Simulation at step <stepCounter>
<totalPeopleHealthy> people healthy, <totalPeopleIll> infected
Cities:
<toString_of_city1>
<toString_of_city2>
...
<toString_of_cityN>
```

Die Städte sollen in alphabetischer Reihenfolge aufgelistet werden.

Benutzen Sie `System.lineSeparator()` für Zeilenumbrüche.

Am Ende des String soll sich kein Zeilenumbruch mehr befinden.